

Estudio: PythonChatGPT

Índice

1. [¿Qué hace este proyecto?](#)
 2. [Arquitectura general](#)
 3. [Flujo de ejecución](#)
 4. [Desglose de cada archivo](#)
 5. [Dependencias externas](#)
 6. [Conceptos clave](#)
 7. [Posibles mejoras](#)
-

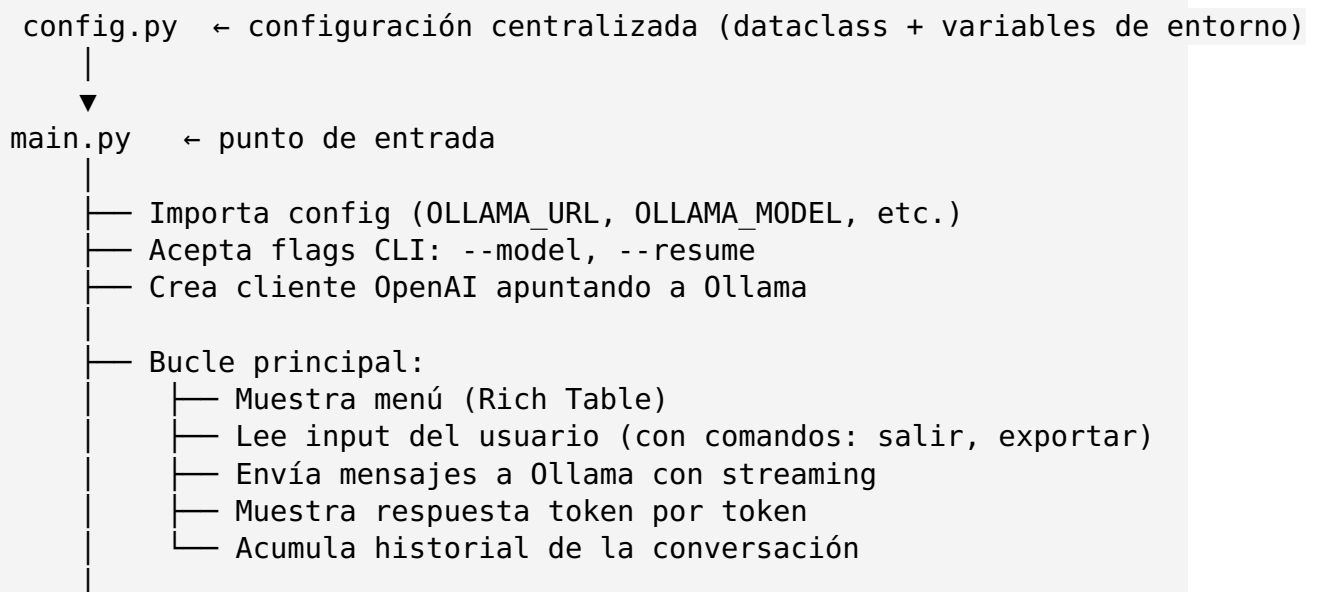
1. ¿Qué hace este proyecto?

Es un **chatbot de terminal** (CLI) que se conecta a un modelo de lenguaje ejecutándose localmente a través de [Ollama](#). El usuario escribe preguntas y el modelo responde en tiempo real (streaming), manteniendo el contexto de toda la conversación.

Tecnologías involucradas:

- Ollama (servidor local de LLMs)
 - API compatible con OpenAI (el SDK oficial de OpenAI)
 - Terminal interactiva con formato visual (Rich + Typer)
-

2. Arquitectura general



```
└─ Al salir:
    └─ Guarda conversación en conversations/*.json
    └─ Exporta a Markdown en conversations/export_*.md

conversations/ ← archivos JSON y MD de conversaciones guardadas
tests/        ← tests con pytest
```

Estructura actual del proyecto:

```
PythonChatGPT/
├─ main.py           ← Lógica principal
├─ config.py         ← Configuración (dataclass)
├─ config.example.py ← Documentación de variables
├─ requirements.txt  ← Dependencias
├─ README.md         ← Documentación
├─ ESTUDIO.md        ← Este documento
├─ ESTUDIO.pdf       ← Versión PDF
├─ .gitignore        ← Archivos ignorados
├─ conversations/    ← Conversaciones guardadas aquí
├─ tests/
│   └─ __init__.py
│   └─ test_main.py  ← 4 tests unitarios
```

3. Flujo de ejecución

3.1 Importaciones (líneas 1-9)

```
from openai import OpenAI           # SDK oficial de OpenAI
from openai import APIConnectionError # Excepción específica
import typer                         # Framework para crear CLIs
from rich import print               # print con colores y formato
from rich.table import Table         # Tablas formateadas en terminal
from datetime import datetime        # Timestamps para archivos
import json                          # Serializar conversaciones
from pathlib import Path              # Manejo de rutas

from config import config            # Configuración centralizada
```

openai: SDK oficial. Aunque está diseñado para la API de OpenAI, también funciona con servidores compatibles como Ollama (que expone una API idéntica). Esto permite cambiar entre Ollama local y OpenAI en la nube sin tocar el código.

typer: Framework para crear aplicaciones de línea de comandos. Convierte una función Python en un comando CLI con `--help` automático.

rich: Biblioteca para terminales bonitas. Permite colores, tablas, markdown, barras de progreso, etc.

3.2 Configuración desde `config.py`

```
# config.py
import os
from dataclasses import dataclass, field

@dataclass
class Config:
    base_url: str = field(default_factory=lambda: os.getenv("OLLAMA_URL", "http"))
    api_key: str = field(default_factory=lambda: os.getenv("OLLAMA_API_KEY", "o"))
    model: str = field(default_factory=lambda: os.getenv("OLLAMA_MODEL", "llama"))

config = Config()
```

```
# main.py
from config import config
client = OpenAI(base_url=config.base_url, api_key=config.api_key)
```

@dataclass : Decorador que genera automáticamente `__init__` , `__repr__` , etc. Los campos se definen con anotaciones de tipo.

field(default_factory=...) : Permite que el valor por defecto se calcule en tiempo de ejecución (necesario porque `os.getenv` debe ejecutarse cuando se crea la instancia, no cuando se define la clase).

Por qué `api_key="ollama"` : Ollama no requiere autenticación real, pero el SDK de OpenAI exige una API Key. Se pasa cualquier valor.

Ventaja: La configuración está en un solo archivo. Si en el futuro se necesitan más variables (ej: `OLLAMA_TIMEOUT`), se agregan en `config.py` sin tocar `main.py` .

3.3 Funciones auxiliares (líneas 14-48)

3.3.1 `_load_last_conversation()` — Cargar conversación previa

```
def _load_last_conversation() -> list[dict]:
    if not CONVERSATIONS_DIR.exists():
        return []
    files = sorted(CONVERSATIONS_DIR.glob("*.json"), reverse=True)
    if not files:
        return []
    return json.loads(files[0].read_text())
```

Path.glob("*.json") : Busca todos los archivos `.json` en el directorio.

sorted(..., reverse=True) : Ordena por nombre (que incluye timestamp) y toma el más reciente.

json.loads(files[0].read_text()) : Lee el archivo JSON y lo convierte a lista de diccionarios.

Propósito: Permitir `--resume` para retomar la última conversación.

3.3.2 `_save_conversation()` — Guardar conversación

```
def _save_conversation(messages: list[dict]) -> Path:
    CONVERSATIONS_DIR.mkdir(exist_ok=True)
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    path = CONVERSATIONS_DIR / f"chat_{timestamp}.json"
    path.write_text(json.dumps(messages, indent=2, ensure_ascii=False))
    return path
```

strftime("%Y%m%d_%H%M%S") : Genera un timestamp como `20260606_221530` .

json.dumps(..., indent=2, ensure_ascii=False) : Serializa a JSON con formato legible y soporte para caracteres UTF-8 (acentos, ñ, etc.).

3.3.3 `_export_markdown()` — Exportar a Markdown

```
def _export_markdown(messages: list[dict]) -> Path:
    lines = ["# Conversación - PythonChatGPT\n"]
    for msg in messages:
        role = {"system": "🔧 Sistema", "user": "👤 Tú", "assistant": "🤖 Asistente"}
        lines.append(f"## {role}\n")
        lines.append(f"{msg['content']}\n")
    path = CONVERSATIONS_DIR / f"export_{datetime.now().strftime('%Y%m%d_%H%M%S')}.md"
    path.write_text("\n".join(lines))
    return path
```

.get(msg["role"], msg["role"]) : Usa el emoji correspondiente según el rol. Si el rol no está en el diccionario, usa el valor original.

Propósito: Generar un archivo Markdown legible para compartir o guardar respuestas importantes.

3.4 La función main() (líneas 50-102)

3.4.1 Firma con Typer (líneas 50-53)

```
def main(
    model: str = typer.Option(config.model, "--model", "-m", help="Modelo de OL")
```

```
    resume: bool = typer.Option(False, "--resume", "-r", help="Retomar la última conversación")
) -> None:
```

typer.Option() : Convierte el parámetro en una opción de línea de comandos:

- "--model" / "-m" — ambas flags funcionan
- config.model — valor por defecto (viene de variable de entorno o default)

Prioridad: Flag CLI → variable de entorno → default en Config.

3.4.2 Cabecera (líneas 55-62)

```
client = OpenAI(base_url=config.base_url, api_key=config.api_key)

print("[bold yellow]Chat-bot con IA (Ollama)[/bold yellow]")
print(f"  Modelo: [cyan]{model}[/cyan]")
print(f"  Servidor: [cyan]{config.base_url}[/cyan]")

table = Table("Opciones", "Descripción")
table.add_row("salir", "Terminar la conversación")
table.add_row("exportar", "Exportar la conversación a Markdown")
print(table)
```

Ahora el cliente se crea DENTRO de `main()` para poder usar el modelo del flag.

Se agregó el comando `exportar` a la tabla de opciones.

3.4.3 Inicializar mensajes (líneas 64-72)

```
if resume:
    messages = _load_last_conversation()
    if messages:
        print("[green]Conversación retomada.[/green]")
    else:
        print("[yellow]No hay conversaciones previas. Empezando nueva.[/yellow]")
        messages = [{"role": "system", "content": "Eres un asistente muy útil."}]
else:
    messages = [{"role": "system", "content": "Eres un asistente muy útil."}]
```

Si `--resume` está activo, carga la última conversación guardada. Si no hay, empieza una nueva.

3.4.4 Bucle principal (líneas 74-101)

```
while True:
    content = input("\n[bold cyan]Escribe tu pregunta: [/bold cyan]")
    if content == "salir":
```

```

        break
    if content == "exportar":
        md_path = _export_markdown(messages)
        print(f"[green]Conversación exportada a: {md_path}[/green]")
        continue

    messages.append({"role": "user", "content": content})

    try:
        response = client.chat.completions.create(
            model=model,
            messages=messages,
            stream=True,
        )
        ...
    except APIConnectionError:
        print("\n[red]Error: No se pudo conectar a Ollama.[/red]")
        print("[yellow]Asegúrate de que Ollama esté corriendo:[/yellow]")
        print("  ollama serve")
        break

```

Comando `exportar` : Exporta la conversación actual sin salir del programa.

`model=model` (antes `MODEL`): Usa el parámetro de la función, que puede venir del flag CLI.

3.4.5 Al salir (líneas 103-105)

```

_save_conversation(messages)
md_path = _export_markdown(messages)
print(f"\n[green]Conversación guardada. Exportada a: {md_path}[/green]")

```

Siempre se guarda y exporta al salir, incluso si no se usó `--resume` .

3.5 Punto de entrada (línea 108)

```

if __name__ == "__main__":
    typer.run(main)

```

`if __name__ == "__main__"` : Patrón estándar de Python. El código dentro solo se ejecuta cuando el archivo se corre directamente (`python main.py`), no cuando se importa desde otro módulo.

`typer.run(main)` : Toma la función `main()` y la convierte en un comando CLI. Typer automáticamente:

- Genera ayuda con `--help`
- Maneja argumentos de línea de comandos (`--model` , `--resume`)

- Convierte el `bool` de `resume` en flag (`--resume` presente = `True` , ausente = `False`)
-

4. Desglose de cada archivo

4.1 `main.py`

Propósito: Archivo ejecutable principal. Contiene la lógica de la CLI, el bucle de conversación y las funciones auxiliares. **Líneas:** 108 **Responsabilidades:**

- Importar configuración desde `config.py` (línea 10)
- Crear cliente OpenAI (línea 55)
- Funciones auxiliares: `_load_last_conversation` , `_save_conversation` , `_export_markdown` (líneas 14-48)
- Interfaz de usuario con Rich y Typer (líneas 57-62)
- Bucle de conversación con streaming (líneas 74-101)
- Manejo de errores `APIConnectionError` (líneas 96-100)
- Guardado y exportación al salir (líneas 103-105)

4.2 `config.py`

Propósito: Centralizar toda la configuración del proyecto en un dataclass. **Líneas:** 12 **Responsabilidades:**

- Leer variables de entorno con valores por defecto
- Proveer un objeto `Config` con `base_url` , `api_key` , `model`
- Ser importable desde `main.py` y desde `tests/`

Ventaja sobre tener la config en main.py: Si se agregan más variables (timeout, temperatura del modelo, etc.), solo se modifica `config.py` .

4.3 `requirements.txt`

Propósito: Lista de dependencias para `pip install` .

- `openai>=1.0.0`: SDK para comunicarse con la API.
- `typer>=0.9.0`: Framework CLI.
- `rich>=13.0.0`: Formato de terminal.

4.4 `config.example.py`

Propósito: Documentar las variables de entorno disponibles. **Uso:** Copiar a `.env` o exportar las variables manualmente.

4.5 `.gitignore`

Propósito: Evitar que archivos locales se suban a Git. **Excluye:**

- `.venv/` — entorno virtual
- `config.py` — claves reales
- `__pycache__/` y `*.pyc` — archivos compilados

- `conversations/*.json` y `conversations/*.md` — datos de usuario
- `.vscode/` y `.idea/` — configuraciones de IDE

4.6 README.md

Propósito: Documentación del proyecto para quien visite el repositorio.

4.7 tests/test_main.py

Propósito: Tests unitarios con pytest (4 tests). **Funcionalidad probada:**

- `test_config_defaults` : Verifica valores por defecto de Config
- `test_config_from_env` : Verifica que las variables de entorno sobrescriban
- `test_save_and_load_conversation` : Verifica guardado y carga JSON
- `test_export_markdown` : Verifica que el Markdown incluya roles y contenido

Estructura de cada test: Usa `monkeypatch` para simular variables de entorno y `TemporaryDirectory` para aislar archivos temporales.

5. Dependencias externas

5.1 Ollama (no está en requirements.txt)

Ollama es un **servidor** que debe instalarse aparte. No es una librería Python.

Instalación: `curl -fsSL https://ollama.ai/install.sh | sh`

Inicio: `ollama serve` (corre en segundo plano en `http://localhost:11434`)

Modelos: Se descargan con `ollama pull <nombre>` (ej: `llama3.2:1b` , `llama3.1:8b` , `mistral` , `codellama`)

5.2 openai (librería Python)

SDK oficial de OpenAI. Su función principal es `client.chat.completions.create()` que:

1. Conecta a `base_url/v1/chat/completions` (endpoint REST)
2. Envía un JSON con `model` , `messages` , `stream`
3. Recibe la respuesta (completa o en streaming)

Aunque Ollama no es OpenAI, implementa el mismo protocolo REST, por lo que el SDK funciona sin modificaciones.

5.3 typer

Construido sobre Click. Convierte funciones Python en comandos CLI:

- `typer.run(main)` — ejecuta `main()` como comando
- `@app.command()` — para múltiples comandos

- Soporta argumentos, opciones, `--help` automático

5.4 rich

Provee:

- `print()` mejorado con formato BBCode (`[bold]` , `[red]` , `[yellow]` , etc.)
 - `Table()` con columnas, bordes, alineación
 - `Panel()` , `Progress()` , `Markdown()` , `Syntax()`
-

6. Conceptos clave

6.1 Streaming vs no-streaming

| Sin streaming | Con streaming | |---|---| | Esperas todo el tiempo de generación | Ves los tokens en tiempo real | | La respuesta llega completa de una vez | La respuesta llega fragmentada | | Menor complejidad de código | Mayor complejidad (bucle) | | Peor experiencia de usuario | Mejor experiencia de usuario |

6.2 El protocolo de mensajes

El formato de mensajes es un estándar en APIs de LLMs:

```
[
  {"role": "system", "content": "Eres un asistente..."},
  {"role": "user", "content": "¿Qué es Python?"},
  {"role": "assistant", "content": "Python es un lenguaje..."},
  {"role": "user", "content": "¿Y qué es un decorador?"}
]
```

Cada interacción agrega **dos mensajes**: la pregunta del usuario y la respuesta del asistente. El modelo usa todo el historial para generar contexto.

6.3 Variables de entorno

`OLLAMA_URL=http://localhost:11434/v1` — apunta a Ollama local. Si se cambia a `https://api.openai.com/v1` y se pone una API key real, el mismo código funciona con OpenAI.

7. Historial de mejoras implementadas

✓ Mejora 1 — Configuración separada en `config.py`

Archivos: `config.py` (nuevo), `main.py` (modificado) **Qué cambió:** Se centralizaron `base_url` , `api_key` , `model` en un dataclass. **Por qué:** Antes la

configuración estaba mezclada con la lógica. Ahora se puede cambiar desde un solo lugar y es fácil de testear.

✓ Mejora 2 — Persistencia de conversaciones en JSON

Archivos: `main.py` (funciones `_save_conversation`, `_load_last_conversation`) **Qué cambió:** Cada conversación se guarda en `conversations/chat_YYYYMMDD_HHMMSS.json`. **Por qué:** Permite retomar conversaciones con `--resume` y no perder el historial.

✓ Mejora 3 — Selector de modelos por flag CLI

Archivos: `main.py` (parámetro `model` en `main()`) **Qué cambió:** `python main.py --model llama3.1:8b` sobrescribe el modelo. **Por qué:** Antes solo se podía cambiar via variable de entorno. Ahora es más rápido y no requiere reiniciar la terminal.

✓ Mejora 4 — Exportación a Markdown

Archivos: `main.py` (función `_export_markdown`) **Qué cambió:** Comando `exportar` durante la conversación, y export automático al salir. **Por qué:** Permite guardar respuestas importantes en un formato legible y compatible.

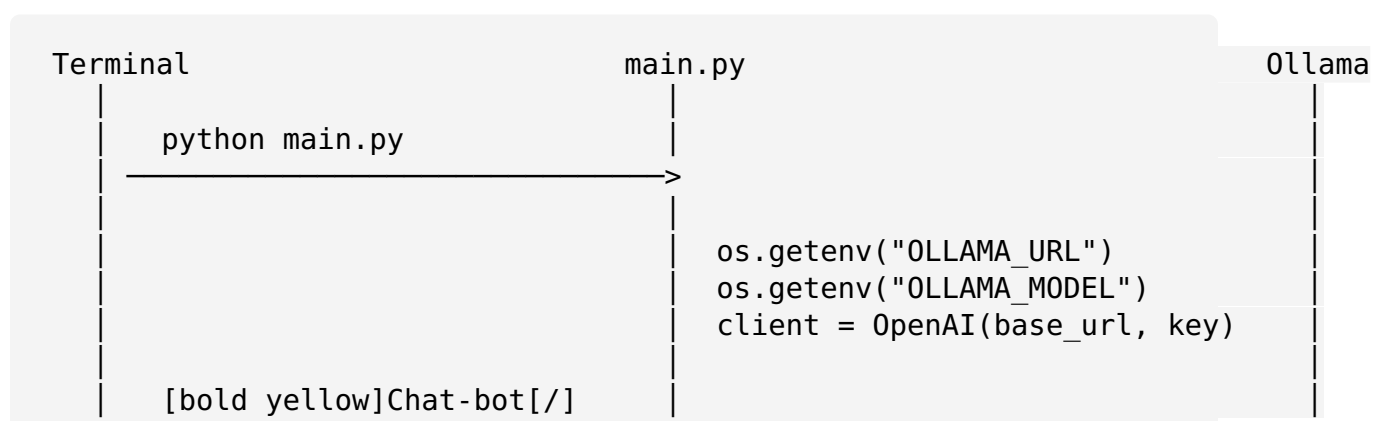
✓ Mejora 5 — Tests con pytest

Archivos: `tests/test_main.py` (4 tests) **Qué cambió:** Se agregaron tests unitarios para config, persistencia y exportación. **Por qué:** Los tests son señal de profesionalismo. Verifican que el código funciona y previenen regresiones.

▶ Próximas mejoras posibles

| Prioridad | Mejora | Esfuerzo | |---|---|---| | Alta | Múltiples conversaciones con nombres | Medio | | Alta | Soporte para OpenAI además de Ollama (flag `--provider`) | Medio | | Media | Interfaz web con Gradio | Alto | | Media | Plugins (ej: ejecutar código, buscar web) | Alto | | Baja | RAG (búsqueda en documentos propios) | Muy alto |

8. Diagrama visual del flujo



Salir	salir del...
-------	--------------

"Escribe tu pregunta:"
< usuario escribe texto

"Python" (aparece en vivo)

" es"

" un"

"Escribe tu pregunta:"
< usuario escribe "salir"

Fin del programa

```
POST /v1/chat/completions
{model, messages, stream=True}
```

<chunk 1: "Python"> <— HTTP 200

<chunk 2: " es">

<chunk 3: " un">

...

<chunk N: finish_reason="stop">